STEP SEM-3 · Week 8

Category C Practice – Inheritance and Polymorphism

Solutions: coding problems, quiz answers, and concept answers

Part	Contents
1	Coding Questions – Problems 1 to 5 (Python, with tested sample output)
2	Quiz Questions – Questions 1 to 10 with answers and reasons
3	Concept Questions – Questions 1 to 10

Design used in every coding solution: an abstract base class declares one operation, each subclass overrides it with its own rule, and the main loop works on a list of base-type objects and calls that operation without any if/else on the type. A dictionary maps the input keyword to a class, so a new type only needs a new class and one dictionary entry.

Part 1 – Coding Questions

Problem 1: Payment System Fee Calculation

Card adds 2%, Wallet adds 1%, Bank Transfer adds nothing. Each class overrides `final_amount()`; the loop just calls it and accumulates the total.

Solution (Python)

```
import sys
from abc import ABC, abstractmethod


class Payment(ABC):
    def __init__(self, amount):
        self.amount = amount

    @abstractmethod
    def final_amount(self):
        """Amount after applying this payment method's processing fee."""


class CardPayment(Payment):
    def final_amount(self):
        return self.amount * 1.02          # 2% fee


class WalletPayment(Payment):
    def final_amount(self):
        return self.amount * 1.01          # 1% fee


class BankTransferPayment(Payment):
    def final_amount(self):
        return self.amount                 # no fee


# Registry: adding a new payment type only needs a new class + one entry here.
PAYMENT_TYPES = {
    "CARD": CardPayment,
    "WALLET": WalletPayment,
    "BANKTRANSFER": BankTransferPayment,
}


def main():
    data = sys.stdin.read().split("\n")
    n = int(data[0])
    payments = []                          # polymorphic collection of Payment
    for line in data[1:n + 1]:
        kind, amount = line.split()
        payments.append((kind.upper(), PAYMENT_TYPES[kind.upper()](float(amount))))

    total = 0.0
    for kind, payment in payments:         # uniform processing, no type checks
        result = payment.final_amount()
        total += result
        print(f"{kind}: {result:.2f}")
    print(f"Total: {total:.2f}")


if __name__ == "__main__":
    main()
```

Sample input

```
3
CARD 1000
WALLET 500
BANKTRANSFER 2000
```

Output of the code above

```
CARD: 1020.00
WALLET: 505.00
BANKTRANSFER: 2000.00
Total: 3525.00
```

Problem 2: Library Item Due Date Calculator

The base class owns the shared `due_date()` logic. Subclasses only override `loan_days()` (14, 7 and 3). Titles are read by splitting off the type keyword and stripping the quotes, so titles with spaces work.

Solution (Python)

```
import sys
from abc import ABC, abstractmethod
from datetime import date, timedelta

CURRENT_DATE = date(2023, 10, 26)


class LibraryItem(ABC):
    def __init__(self, title):
        self.title = title

    @abstractmethod
    def loan_days(self):
        """Borrowing period in days for this item type."""

    def due_date(self):                    # shared behaviour, inherited as-is
        return CURRENT_DATE + timedelta(days=self.loan_days())


class Book(LibraryItem):
    def loan_days(self):
        return 14


class DVD(LibraryItem):
    def loan_days(self):
        return 7


class Magazine(LibraryItem):
    def loan_days(self):
        return 3


ITEM_TYPES = {"BOOK": Book, "DVD": DVD, "MAGAZINE": Magazine}


def main():
    lines = sys.stdin.read().split("\n")
    n = int(lines[0])
    items = []
    for line in lines[1:n + 1]:
        kind, title = line.strip().split(None, 1)
        title = title.strip().strip('"')
        items.append(ITEM_TYPES[kind.upper()](title))

    for item in items:                     # uniform processing
```

```
        print(f"{item.title}: {item.due_date().strftime('%Y-%m-%d')}")


if __name__ == "__main__":
    main()
```

Sample input

```
3
BOOK "1984"
DVD "The Matrix"
MAGAZINE "Forbes Issue 500"
```

Output of the code above

```
1984: 2023-11-09
The Matrix: 2023-11-02
Forbes Issue 500: 2023-10-29
```

Problem 3: Delivery Fee Calculator

Each delivery class overrides `fee()` with its own base fee and per-kg and per-km rates; International also adds the customs fee.

Note on the sample output: the problem's expected output shows EXPRESS 29.00, INTERNATIONAL 155.00 and Total 199.00. The written business rules give EXPRESS = 15 + 1.00 x 5 + 0.20 x 20 = 24.00 and INTERNATIONAL = 25 + 2.00 x 20 + 0.50 x 100 + 30 = 145.00, so the total is 184.00. STANDARD (15.00) agrees. The code follows the written rules, so its output differs from the printed sample for those two lines. If your judge expects 29.00 and 155.00, the rates in the problem statement are probably misprinted and you would need the real rates.

Solution (Python)

```
import sys
from abc import ABC, abstractmethod


class Delivery(ABC):
    def __init__(self, weight, distance, extra=0.0):
        self.weight = weight
        self.distance = distance
        self.extra = extra

    @abstractmethod
    def fee(self):
        """Delivery fee according to this delivery type's pricing rules."""


class StandardDelivery(Delivery):
    def fee(self):
        return 5 + 0.50 * self.weight + 0.10 * self.distance


class ExpressDelivery(Delivery):
    def fee(self):
        return 15 + 1.00 * self.weight + 0.20 * self.distance


class InternationalDelivery(Delivery):
    def fee(self):
        return 25 + 2.00 * self.weight + 0.50 * self.distance + self.extra


DELIVERY_TYPES = {
    "STANDARD": StandardDelivery,
```

```
    "EXPRESS": ExpressDelivery,
    "INTERNATIONAL": InternationalDelivery,
}


def main():
    lines = sys.stdin.read().split("\n")
    n = int(lines[0])
    deliveries = []
    for line in lines[1:n + 1]:
        parts = line.split()
        kind = parts[0].upper()
        nums = [float(x) for x in parts[1:]]
        deliveries.append((kind, DELIVERY_TYPES[kind](*nums)))

    total = 0.0
    for kind, delivery in deliveries:      # uniform processing
        fee = delivery.fee()
        total += fee
        print(f"{kind}: {fee:.2f}")
    print(f"Total: {total:.2f}")


if __name__ == "__main__":
    main()
```

Sample input

```
3
STANDARD 10 50
EXPRESS 5 20
INTERNATIONAL 20 100 30
```

Output of the code above

```
STANDARD: 15.00
EXPRESS: 24.00
INTERNATIONAL: 145.00
Total: 184.00
```

Problem 4: Examination Question Grader

MCQ and TF award full points on an exact match. Essay counts how many of the comma-separated keywords appear in the student answer (case-insensitive): 2 or more gives 75%, one gives 50%, none gives 0. Quoted fields are extracted with a regular expression.

Solution (Python)

```
import re
import sys
from abc import ABC, abstractmethod


class Question(ABC):
    def __init__(self, text, correct, student, points):
        self.text = text
        self.correct = correct
        self.student = student
        self.points = points

    @abstractmethod
    def score(self):
        """Score earned by the student for this question."""


class MultipleChoiceQuestion(Question):
    def score(self):
```

```
        return float(self.points) if self.student == self.correct else 0.0


class TrueFalseQuestion(Question):
    def score(self):
        return float(self.points) if self.student == self.correct else 0.0


class EssayQuestion(Question):
    def score(self):
        keywords = {k.strip().lower() for k in self.correct.split(",") if k.strip()}
        answer = self.student.lower()
        hits = sum(1 for k in keywords if k in answer)
        if hits >= 2:
            return self.points * 0.75
        if hits == 1:
            return self.points * 0.50
        return 0.0


QUESTION_TYPES = {
    "MCQ": MultipleChoiceQuestion,
    "TF": TrueFalseQuestion,
    "ESSAY": EssayQuestion,
}


def main():
    lines = sys.stdin.read().split("\n")
    n = int(lines[0])
    questions = []
    for line in lines[1:n + 1]:
        kind = line.split()[0].upper()
        text, correct, student = re.findall(r'"([^"]*)"', line)
        points = int(line.split()[-1])
        questions.append((kind, QUESTION_TYPES[kind](text, correct, student, points)))

    total = 0.0
    for kind, question in questions:       # central grader, no type checks
        s = question.score()
        total += s
        print(f"{kind}: {s:.2f}")
    print(f"Total Score: {total:.2f}")


if __name__ == "__main__":
    main()
```

Sample input

```
4
MCQ "What is the capital of France?" "Paris" "Paris" 10
TF "The Earth is flat?" "False" "True" 5
ESSAY "Name two primary OOP principles." "Inheritance, Polymorphism, Encapsulation" "Polymorphism is one." 20
ESSAY "Describe abstraction and composition." "Abstraction, Composition" "I talked about abstraction." 15
```

Output of the code above

```
MCQ: 10.00
TF: 0.00
ESSAY: 10.00
ESSAY: 7.50
Total Score: 27.50
```

Problem 5: Public Transport Fare Calculator

Bus is capped at $10 with `min()`, Train is linear, and Metro multiplies by the peak-hour factor.

Solution (Python)

```
import sys
from abc import ABC, abstractmethod


class Journey(ABC):
    def __init__(self, distance, factor=1.0):
        self.distance = distance
        self.factor = factor

    @abstractmethod
    def fare(self):
        """Fare for this journey according to the transport type's rules."""


class BusJourney(Journey):
    def fare(self):
        return min(2 + 0.10 * self.distance, 10.0)      # capped at $10


class TrainJourney(Journey):
    def fare(self):
        return 3 + 0.15 * self.distance


class MetroJourney(Journey):
    def fare(self):
        return (1.50 + 0.20 * self.distance) * self.factor


JOURNEY_TYPES = {
    "BUS": BusJourney,
    "TRAIN": TrainJourney,
    "METRO": MetroJourney,
}


def main():
    lines = sys.stdin.read().split("\n")
    n = int(lines[0])
    journeys = []
    for line in lines[1:n + 1]:
        parts = line.split()
        kind = parts[0].upper()
        nums = [float(x) for x in parts[1:]]
        journeys.append((kind, JOURNEY_TYPES[kind](*nums)))

    total = 0.0
    for kind, journey in journeys:         # uniform processing
        fare = journey.fare()
        total += fare
        print(f"{kind}: {fare:.2f}")
    print(f"Total: {total:.2f}")


if __name__ == "__main__":
    main()
```

Sample input

```
3
BUS 15
TRAIN 50
METRO 10 1.5
```

Output of the code above

```
BUS: 3.50
TRAIN: 10.50
METRO: 5.25
Total: 19.25
```

Part 2 – Quiz Questions

#	Question	Answer	Why
1	Repeated if-else checking document type before render()	**C – Polymorphism**	Call render() through a Document reference and let each subclass supply its own version. The conditionals disappear.
2	Genuine 'is-a' relationships (select all)	**A, B, C**	Car is a Vehicle, Rectangle is a Shape, and DatabaseConnection is a NetworkResource are all valid. D (HelperUtility contains a Calculator) is 'has-a', which calls for composition.
3	What picks the right startEngine() for each object in a Vehicle list?	**C – Runtime method dispatch**	The method is chosen from the actual object's class at runtime. Static binding, compile-time polymorphism and overloading are all resolved at compile time.
4	Shape list, calculateArea() called on each	**C – Inheritance-based polymorphism**	One interface, different behavior depending on the real object. Overloading is same name with different parameters, and composition is 'has-a'.
5	LibraryItem, Book and DVD loan periods (select all)	**A, C, D**	A: Book extends inherited behavior. C: LibraryItem defines what all items share. D: a DVD can be used through a LibraryItem reference. B is wrong because DVD sets its own fixed period, which is specialized behavior, not shared behavior.
6	Animal, Dog and Cat makeSound() (select all)	**A, C**	A: an Animal reference to a Dog runs Dog's version. C: Dog's makeSound() is overridden behavior. B is wrong because Animal's version is the general one. D is wrong because derived classes can add new members freely.
7	Benefit of Payment references when adding WalletPayment	**C**	The processor iterates over Payment references, so a new subclass works without changing the processor's loop.
8	Why inheritance only for code reuse between unrelated classes is bad	**A**	It creates tight coupling and a false 'is-a' relationship, which makes the design rigid. Performance, debugging and interface claims in B, C and D are not the real reason.
9	Advantages in the Notification design (select all)	**A, C**	A: a generic sender works with any Notification. C: a new channel such as InAppNotification needs no change to the sender. B and D are wrong because each subclass overrides send() with its own channel logic.

#	Question	Answer	Why
10	Benefits of polymorphic collections (select all)	**A, B, C**	They simplify iteration, allow uniform processing of specialized objects, and avoid explicit casting in common loops. D is the opposite of the truth: new derived types are easier to add.

Part 3 – Concept Questions

Question 1

How inheritance lets specialized classes reuse and extend behavior of a base class, with a business example.

Inheritance lets a derived class start with everything the base class already has: its fields and methods. The derived class can use these as they are (reuse), add new members (extend), or redefine a method to suit its own needs (modify). Common code is written once in the base class and every subclass gets it for free.

Example (banking): `BankAccount` has `balance`, `deposit()` and `withdraw()`. `SavingsAccount` reuses both and **extends** them with `add_interest()`. `CurrentAccount` **modifies** `withdraw()` to allow an overdraft limit. Deposit logic exists in one place only.

Question 2

The 'is-a' relationship and why identifying it correctly matters.

An 'is-a' relationship means the derived class is a specialized kind of the base class, so an object of the derived class can be used anywhere the base class is expected without breaking the program (the Liskov substitution idea). A Car is a Vehicle; a SavingsAccount is a BankAccount.

Why it matters:

- If the relationship is false (for example, Stack extends ArrayList), the subclass inherits methods that make no sense for it and callers can misuse it.
- Code written for the base type will behave wrongly or crash when given the derived type.
- It creates tight coupling: every change in the base class ripples into unrelated subclasses.
- 'Has-a' relationships (a Car has an Engine) should use composition, not inheritance.

Question 3

Method overriding, illustrated with Employee salary calculation.

Method overriding means a derived class provides its own implementation of a method that is already defined in its base class, using the same name and parameters. The version that runs depends on the actual object's class.

```
class Employee:
    def __init__(self, name):
        self.name = name
    def calculate_salary(self):
        return 0

class FullTimeEmployee(Employee):
    def __init__(self, name, monthly, bonus):
        super().__init__(name)
        self.monthly, self.bonus = monthly, bonus
    def calculate_salary(self):                # overrides
        return self.monthly + self.bonus

class PartTimeEmployee(Employee):
    def __init__(self, name, hours, rate):
        super().__init__(name)
        self.hours, self.rate = hours, rate
    def calculate_salary(self):                # overrides
        return self.hours * self.rate
```

```
staff = [FullTimeEmployee("Asha", 50000, 5000), PartTimeEmployee("Ravi", 80, 300)]
for e in staff:
    print(e.name, e.calculate_salary())         # 55000 and 24000
```

Each employee type calculates pay in its own way, but the payroll loop calls the same method on every object.

Question 4

Runtime polymorphism (dynamic dispatch) with base-type references.

With a base-type reference such as `Vehicle v = new Car()`, the reference type says which methods may be called, but the **actual object** decides which implementation runs. This decision is made while the program runs, not at compile time.

How the system decides: every object knows its own class. In Java and C++ the object carries a hidden pointer to its class's method table (vtable). When a call such as `v.startEngine()` executes, the runtime follows the object to its class, finds the overriding method there, and runs it. If the class has no such override, the search continues up to the parent class. In Python the same effect comes from looking up the method on `type(obj)` and its parents at call time. This is different from overloading, which is resolved at compile time from the argument types.

Question 5

Polymorphic collections: example and advantages.

A polymorphic collection is a list or array typed with a common base class that actually holds objects of different derived classes.

```
shapes = [Circle(3), Rectangle(4, 5), Triangle(6, 2)]   # list of Shape objects
total = 0
for s in shapes:
    total += s.area()      # Circle.area, Rectangle.area, Triangle.area chosen at runtime
```

Advantages:

- One loop processes diverse but related objects uniformly.
- No type checks or casting are needed in the processing code.
- New types can be added to the collection without changing the loop.
- Less duplicated code, so it is easier to read, test and maintain.

Question 6

Polymorphism versus repeated type-based conditional logic.

Conditional approach: the caller checks the type (if kind == 'CARD' ... elif kind == 'WALLET' ...) and runs different code in each branch. **Polymorphic approach:** each class overrides a shared method and the caller just calls it.

- **Adding a type:** conditionals mean editing every if/else chain in the program; polymorphism means adding one class.
- **Risk of bugs:** it is easy to miss one chain and leave a type unhandled; with polymorphism the compiler or interpreter reports a missing override.

- **Organization:** conditionals scatter one type's rules across many places; polymorphism keeps them inside the class.
- **Readability and testing:** a loop with one method call is shorter, and each class can be tested alone.
- **Design principle:** polymorphism follows the open/closed principle (open for extension, closed for modification); chains of conditionals do not.

Because of these points, polymorphism is generally preferred when related objects vary in behavior.

Question 7

Adding new derived types with minimal change, using a new PaymentMethod.

The processing logic depends only on the base type's interface, never on the concrete classes. So adding a new subclass that honors the interface leaves the processing code untouched.

```
class Payment:
    def fee(self): ...

class CardPayment(Payment):  ...
class WalletPayment(Payment): ...

def process(payments):                  # never changes
    return sum(p.fee() for p in payments)

# Later, a new method is introduced:
class UpiPayment(Payment):              # new class only
    def fee(self):
        return self.amount * 0.005

process([CardPayment(100), UpiPayment(200)])   # works immediately
```

Only the new class (plus a registry entry if types are created from input) is added. `process()` and all existing payment classes stay as they were.

Question 8

Inherited behavior versus overridden behavior.

Inherited behavior is a method the derived class uses exactly as written in the base class. **Overridden behavior** is a method the derived class redefines with its own version.

When to inherit: when the base implementation is correct for every subclass. Example: `LibraryItem.due_date()` is the same for all items, so Book, DVD and Magazine inherit it; likewise `Animal.eat()` for Dog and Cat.

When to override: when the rule differs by type. Example: `loan_days()` returns 14 for Book, 7 for DVD and 3 for Magazine; `makeSound()` gives 'Woof' for Dog and 'Meow' for Cat. A subclass can also override and still call `super()` to extend the base behavior rather than replace it.

Question 9

Why inheritance solely for code reuse, without an 'is-a' relationship, is a poor design.

Inheritance is a statement about what a class *is*, not just a way to borrow code. If class B extends class A only to reuse some of A's methods, the design says something false, and it causes problems:

- **Tight coupling:** B depends on A's internals, so changes in A can break B (the fragile base class problem).

- **Broken substitutability:** B objects can be passed where A is expected but do not behave like A.
- **Polluted interface:** B exposes inherited public methods that make no sense for it.
- **Rigid hierarchy:** unrelated classes get tied together, so restructuring later is hard, and a single-inheritance language allows only one parent.
- **Confusing code:** readers assume a real relationship and make wrong assumptions.

Better: use composition (hold an object and delegate to it) or shared utility functions.

Question 10

VehicleRental system with CarRental and TruckRental: common and specialized behavior.

Common to all rentals: customer, vehicle id, start date, number of days, `base_cost()` (days x daily rate), overdue check, and printing a receipt.

Specialized: CarRental adds an insurance fee and passenger count; TruckRental adds a per-kilometer charge, a load-capacity surcharge and a commercial-license check.

```
class VehicleRental:                        # common behavior
    def __init__(self, customer, days, daily_rate):
        self.customer, self.days, self.daily_rate = customer, days, daily_rate
    def base_cost(self):                    # inherited by all
        return self.days * self.daily_rate
    def total_cost(self):                   # overridden by subclasses
        return self.base_cost()

class CarRental(VehicleRental):
    def __init__(self, customer, days, daily_rate, insurance):
        super().__init__(customer, days, daily_rate)
        self.insurance = insurance
    def total_cost(self):
        return self.base_cost() + self.insurance

class TruckRental(VehicleRental):
    def __init__(self, customer, days, daily_rate, km, per_km):
        super().__init__(customer, days, daily_rate)
        self.km, self.per_km = km, per_km
    def total_cost(self):
        return self.base_cost() + self.km * self.per_km
```

The shared fields and `base_cost()` live once in the base class, each subclass overrides `total_cost()` with its own rule, and a list of `VehicleRental` objects can be totalled with one loop.